数值积分

第二十一章 · 复习

温故知新

PHYB57 · CHAPTER 21

Numerical *Integration*

数值积分 · 中英双语讲解

Exam Review Edition — 复习考试版

壹	记号与误差阶	*Notation & orders*
贰	黎曼和	*Riemann sums*
叁	中点法则	*Midpoint rule*
肆	梯形法则	*Trapezoid rule*
伍	辛普森法则	*Simpson's rule*
陆	实战：∫ sin x	*Worked example*
柒	SciPy 工具箱	*Python tools*
捌	总表与易错点	*Summary & traps*
玖	练习题	*Practice problems*
拾	参考答案	*Answers*
◎	速记卡	*Cheat sheet*

温故而知新，可以为师矣。

——《论语·为政》

Rectangles, trapezoids, parabolas:
a short history of being slightly less wrong.

壹 本章地图与记号

Chapter map, notation & what "order" means

本章只做一件事：**用有限个函数值去逼近定积分** $\int_a^b f(x)\,dx$。所有方法都是"切小段 → 每段用简单图形代替曲线 → 求和"。区别只在于每段用什么图形：矩形、梯形、还是抛物线。图形越"贴"曲线，误差阶越高。

> *One idea, five costumes: chop $[a,b]$ into pieces, replace f on each piece by something you can integrate exactly, and add up. Better local shapes ⇒ higher order of accuracy.*

左/右黎曼和	中点法则	梯形法则	辛普森法则	quad 自适应
Riemann	*Midpoint*	*Trapezoid*	*Simpson*	*adaptive quadrature*
$O(h)$	$O(h^2)$	$O(h^2)$	$O(h^4)$	$\approx$ machine ε
矩形 rectangles		*梯形 trapezoids*	*抛物线 parabolas*	*adaptive · 自适应 →*

❖ 记号约定 *Notation*

区间 $[a,b]$ 被等分为 n **个子区间**，节点为 $x_0 = a < x_1 < \cdots < x_n = b$，共 $n+1$ **个点**，步长 $h = \dfrac{b-a}{n}$。记 $f_i = f(x_i)$，子区间中点 $y_i = \dfrac{x_i + x_{i+1}}{2}$。误差统一定义为 $E = I_{\text{exact}} - I_{\text{approx}}$（与讲义代码 `err = 2 - I` 一致）。

$[a,b]$ is split into n **subintervals** with nodes $x_0, \ldots, x_n$ — that is $n+1$ **points** — and uniform width $h = (b-a)/n$. Midpoints are $y_i = (x_i + x_{i+1})/2$. Error is always **exact minus approximation**, matching the slide code `err = 2 - I`.

考点 · 头号陷阱 *Trap #1*

公式里的 n 与代码里的 `n` 不是同一个东西！讲义公式中 n 是子区间数；而讲义代码 `n = 11` 是点数，所以子区间只有 10 个，`h = (b - a) / (n - 1)` $= \pi/10$。看到"n"先问一句：点数还是区间数？

> *In the formulas n counts **subintervals**; in the code `n = 11` counts **points** (10 subintervals). Always ask: points or intervals?*

❖ "误差阶"到底指什么 *Local vs. global order*

说一个方法"是 $O(h^p)$"，意思是误差 $E \approx C\,h^p$（C 与 h 无关）。于是**步长减半，误差约除以 2^p**：一阶方法误差减半，二阶除以 4，四阶除以 16。

要分清两种误差：**局部误差**（单个子区间上）与**全局误差**（整个 $[a,b]$ 上）。子区间个数是 $n = (b-a)/h$，所以把 n 个局部误差加起来：

$$\underbrace{O(h^{p+1})}_{\text{每段 local}} \times \underbrace{\frac{b-a}{h}}_{\text{段数 \#intervals}} = \underbrace{O(h^{p})}_{\text{全局 global}}$$

全局阶 = 局部阶 − 1。这条“降一阶”规则贯穿全章，考试问 “over a single interval” 还是 “over the whole interval” 时一定要对号入座。

> *Summing $(b-a)/h$ local errors of size $O(h^{p+1})$ costs exactly one power of h: global order = local order − 1. Halving h divides a p-th order error by 2^p.*

贰 黎曼和 *Riemann sums — left and right*

定义 *Definition*

黎曼和用**矩形面积之和**逼近积分：每个矩形宽为 h，高取子区间**左端点**（或**右端点**）的函数值。

A Riemann sum approximates the integral by a **sum of rectangle areas**: width h, height f evaluated at the **left** (or **right**) endpoint of each subinterval.

$$L_n = \sum_{i=0}^{n-1} h\, f(x_i), \qquad R_n = \sum_{i=1}^{n} h\, f(x_i).$$

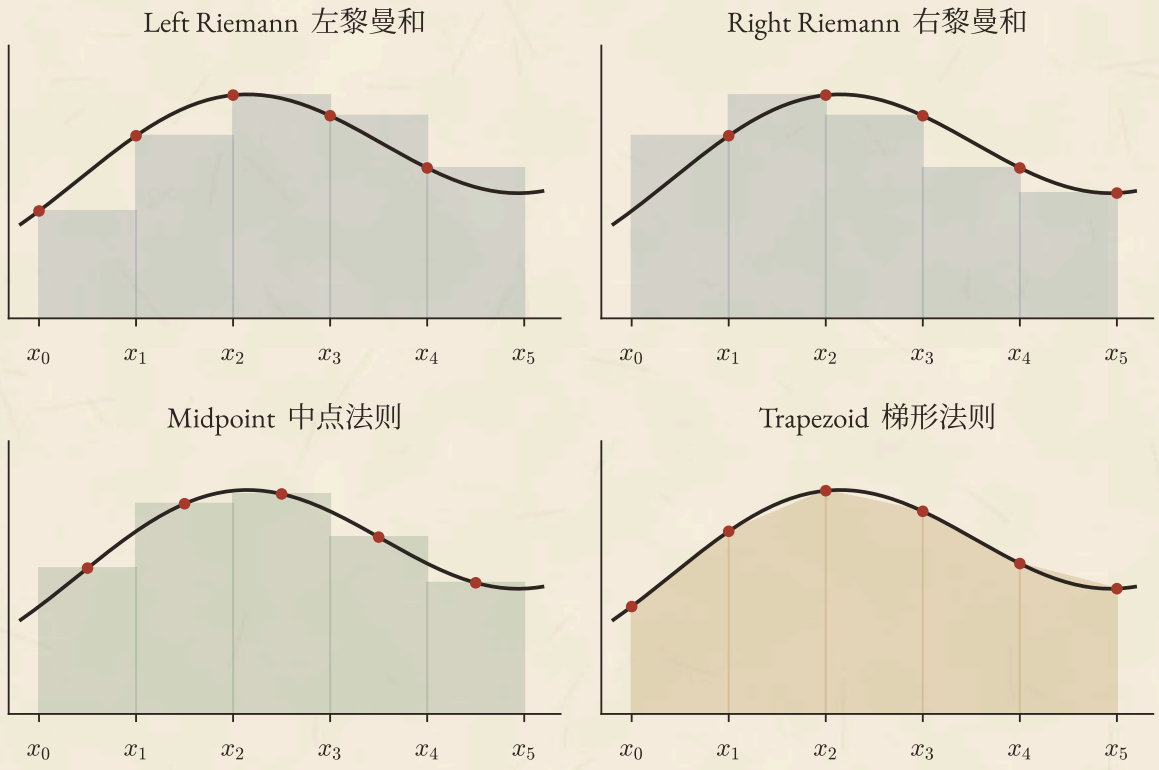


图 1 同一条曲线上的四种“切法”：左黎曼、右黎曼、中点、梯形。红点是被采样的函数值。

Fig. 1 — The same curve sampled four ways; red dots mark where f is evaluated.

❖ 误差推导（泰勒展开） *Error via Taylor series*

推导 *Derivation*

在左端点 x_i 处展开：$f(x) = f(x_i) + f'(x_i)(x - x_i) + \frac{1}{2}f''(\xi)(x - x_i)^2$。逐项积分（$\int_{x_i}^{x_{i+1}} (x - x_i)\,dx = h^2/2$）：

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \;=\; \underbrace{h f(x_i)}_{\text{矩形 rectangle}} \;+\; \underbrace{\frac{h^2}{2} f'(x_i)}_{\text{局部误差主项}} \;+\; O(h^3).$$

所以**局部误差** $O(h^2)$。把 n 段加起来，主项变成一个“黎曼和的黎曼和”：

$$E_L \approx \frac{h}{2}\sum_{i=0}^{n-1} h\,f'(x_i) \approx \frac{h}{2}\int_a^b f'(x)\,dx = \frac{h}{2}\big[f(b) - f(a)\big] \;\Rightarrow\; \textbf{全局 } O(h).$$

右黎曼和同理：$E_R \approx -\frac{h}{2}[f(b) - f(a)]$，符号相反。

结论：黎曼和局部 $O(h^2)$，全局 $O(h)$——**一阶方法**。想提高精度只能把 h 变小，而且很“贵”：误差减半就得把点数翻倍。

Riemann sums are $O(h^2)$ per interval but only $O(h)$ overall — a **first-order** method. Halving the error requires doubling the number of points.

考点 · 一个漂亮的推论 *A neat corollary*

由定义直接相减：$R_n - L_n = h\,[f(b) - f(a)]$。因此**若 $f(a) = f(b)$，左、右黎曼和完全相等**，而且 $O(h)$ 主项 $\frac{h}{2}[f(b) - f(a)]$ 恰好为零——误差会“意外地”变成二阶。

讲义例题 $\int_0^{\pi} \sin x\,dx$ 正是这种情况：$\sin 0 = \sin \pi = 0$。所以讲义输出里 $L = R = 1.9835235\ldots$ 一模一样，并不是巧合，更不是代码写错。

> *Since $R_n - L_n = h[f(b) - f(a)]$, the slides' left and right sums for $\sin x$ on $[0, \pi]$ agree to every digit — and the usual $O(h)$ error term vanishes.*

批 黎曼和是数值积分界的“方便面”：不是最好吃的，但人人都会煮，而且它是后面所有方法的“对照组”。

叁 中点法则 *The midpoint rule*

定义 *Definition*

矩形高度改取**子区间中点** y_i 处的函数值。图形还是矩形，但位置“对称”了。

Same rectangles, but the height is taken at the **midpoint** $y_i = (x_i + x_{i+1})/2$ of each subinterval.

$$M_n = \sum_{i=0}^{n-1} h\,f(y_i), \qquad y_i = \frac{x_i + x_{i+1}}{2}.$$

❖ 为什么只是挪了个位置就升了一阶？ *Why symmetry buys an order*

推导 *Derivation*

这次在**中点** y_i 展开：$f(x) = f(y_i) + f'(y_i)(x-y_i) + \frac{1}{2}f''(y_i)(x-y_i)^2 + \frac{1}{6}f'''(y_i)(x-y_i)^3 + \cdots$

积分区间 $[y_i - \frac{h}{2}, y_i + \frac{h}{2}]$ 关于 y_i 对称，所以**奇次幂项积分为零**：

$$\int (x-y_i)\,dx = 0, \qquad \int (x-y_i)^2\,dx = \frac{h^3}{12}, \qquad \int (x-y_i)^3\,dx = 0.$$

$$\Rightarrow \quad \int_{x_i}^{x_{i+1}} f(x)\,dx = hf(y_i) + \frac{h^3}{24}f''(y_i) + O(h^5).$$

一阶项 $f'(y_i)$ 被对称性**整项消掉**，局部误差从 $O(h^2)$ 跃升为 $O(h^3)$，全局为 $O(h^2)$：

$$E_M \approx \frac{h^2}{24}\big[f'(b) - f'(a)\big].$$

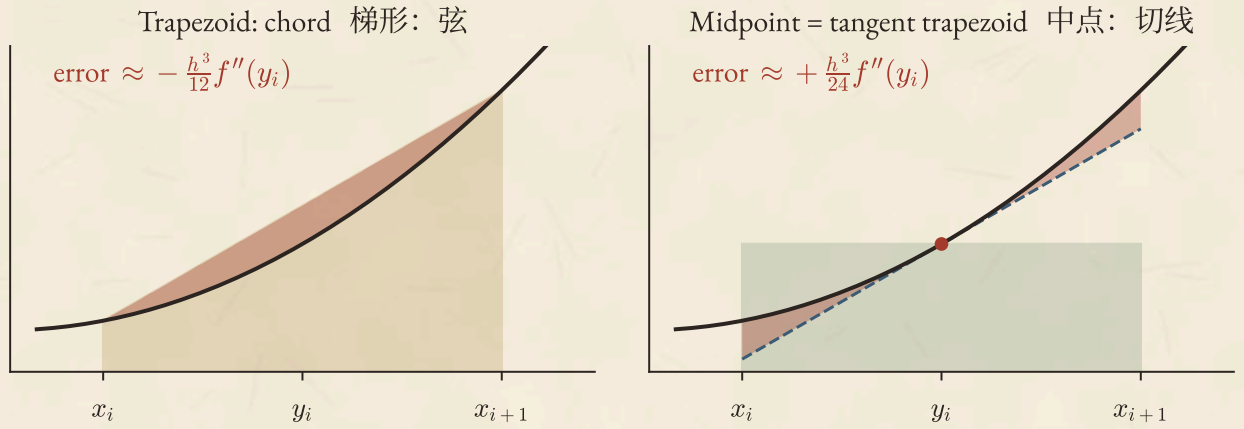


图 2 中点矩形的面积 = 中点切线下的梯形面积。对凸函数（$f'' > 0$），弦在曲线上方（梯形高估），切线在曲线下方（中点低估），且中点误差约为梯形的一半、符号相反。

Fig. 2 — The midpoint rectangle has the same area as the tangent-line trapezoid. Chord over-shoots, tangent under-shoots, and the midpoint error is about −½ of the trapezoid error.

考点 · 局限 *Limitation*

中点法则需要**在中点处取值**。如果你手里只有固定网格上的数据（实验测量、表格），中点上根本没有数据，这个方法就用不了——这正是讲义说的 “data dependency”。

The midpoint rule needs f **at midpoints**. With fixed, tabulated data (measurements on a grid) those values simply do not exist, so the method is unavailable — the slides’ “data dependency”.

肆 梯形法则 *The trapezoid rule*

定义 *Definition*

每个子区间上用连接 (x_i, f_i) 与 (x_{i+1}, f_{i+1}) 的**直线（弦）**代替曲线，面积是一个梯形。

On each subinterval, replace f by the **straight line (chord)** through (x_i, f_i) and (x_{i+1}, f_{i+1}); the area is a trapezoid.

$$\text{Area}_i = \frac{h}{2}(f_i + f_{i+1}), \qquad T_n = \sum_{i=0}^{n-1} \frac{h}{2}(f_i + f_{i+1}) = \frac{h}{2}\Big[f_0 + 2\sum_{i=1}^{n-1} f_i + f_n\Big].$$

❖ 高效写法 *Efficient form*

逐个梯形相加时，每个**内部点**都被相邻两个梯形各用一次，于是要算 $2n$ 次函数值；合并同类项后变成“两端系数 1、内部系数 2”，只需 $n+1$ 次。权重模式：$\frac{h}{2}\,[1, 2, 2, \ldots, 2, 1]$。

> *Interior points are shared by two neighbouring trapezoids; grouping them gives weights* $\frac{h}{2}[1, 2, \ldots, 2, 1]$ *and only* $n+1$ *function evaluations.*

还有一个常被忽略的恒等式：$T_n = \frac{1}{2}(L_n + R_n)$——梯形就是左右黎曼的平均。所以在 $\int_0^{\pi} \sin x\,dx$ 里，既然 $L = R$，那 T 也必然等于它们：讲义三次打印出同一个 1.9835235375094546，原因在此。

❖ 误差推导 *Error analysis*

推导 *Derivation*

在中点 y_i 展开两个端点值（$x_i = y_i - \frac{h}{2}$，$x_{i+1} = y_i + \frac{h}{2}$）：

$$f_i,\ f_{i+1} = f(y_i) \mp \frac{h}{2}f'(y_i) + \frac{h^2}{8}f''(y_i) \mp \frac{h^3}{48}f'''(y_i) + \cdots \ \Rightarrow\ \frac{f_i + f_{i+1}}{2} = f(y_i) + \frac{h^2}{8}f''(y_i) + O(h^4).$$

乘以 h 得梯形面积 $= hf(y_i) + \frac{h^3}{8}f''(y_i) + O(h^5)$；与上一节的精确值 $hf(y_i) + \frac{h^3}{24}f''(y_i)$ 相减：

$$\int_{x_i}^{x_{i+1}} f\,dx - \frac{h}{2}(f_i + f_{i+1}) = \Big(\frac{1}{24} - \frac{1}{8}\Big)h^3 f''(y_i) = -\frac{h^3}{12}f''(y_i) + O(h^5).$$

局部 $O(h^3)$，全局 $O(h^2)$，主项 $E_T \approx -\dfrac{h^2}{12}\big[f'(b) - f'(a)\big]$。对比中点：$E_M \approx -\frac{1}{2}E_T$。

批　讲义说梯形法则是一阶的——这对梯形有点不公平。它和中点法则是“同一届”的二阶生，只是坐在对面、误差符号相反、个头大一倍。

直观理解：梯形法则对**线性函数精确**（弦就是函数本身）；曲率 f'' 越大，弦偏离曲线越多，误差越大——此时应考虑辛普森法则。

勘误 · 讲义第 7 页 *Erratum, slide 7*

讲义第 7 页写道：梯形法则 "error per interval is $O(h^2)$ ... cumulative error becomes $O(h)$"。**这低估了梯形法则整整一阶**。正确结论是**局部** $O(h^3)$、**全局** $O(h^2)$，证据有三：

1. **推导：** 上面三行泰勒展开，一阶项因对称性抵消。
2. **数值：** 对 $\int_0^1 e^x\,dx$，点数 5→9→17 时误差 $8.94\times10^{-3}\to 2.24\times10^{-3}\to 5.59\times10^{-4}$，每次步长减半误差**除以 4.00**（一阶方法只会除以 2）。
3. **自洽：** 讲义第 3 页说中点法则是 $O(h^2)$；而梯形误差主项恰为中点的 -2 倍，两者必然同阶。

> *The slide's "$O(h^2)$ per interval / $O(h)$ overall" is one order too pessimistic. Correct: $O(h^3)$ local, $O(h^2)$ global. On an exam, state the correct order and show the three-line Taylor argument; if in doubt, ask your TA which convention is graded.*

伍 辛普森法则 *Simpson's rule*

定义 *Definition*

每**两个相邻子区间**（三个点 x_{i-1}, x_i, x_{i+1}）上用一条**抛物线**（二次拉格朗日插值多项式）代替曲线，再精确积分这条抛物线。

Over each **pair of subintervals** (three points), fit the **parabola** (quadratic Lagrange interpolant) through the three samples and integrate it exactly.

$$\int_{x_{i-1}}^{x_{i+1}} f\,dx \approx \frac{h}{3}\left(f_{i-1}+4f_i+f_{i+1}\right), \qquad S_n=\frac{h}{3}\Big[f_0+4\sum_{i\ \text{odd}} f_i+2\sum_{i\ \text{even, interior}} f_i+f_n\Big].$$

要求 n **为偶数**（子区间成对），即**点数为奇数**。权重模式：$\frac{h}{3}\,[1,4,2,4,2,\ldots,4,1]$。

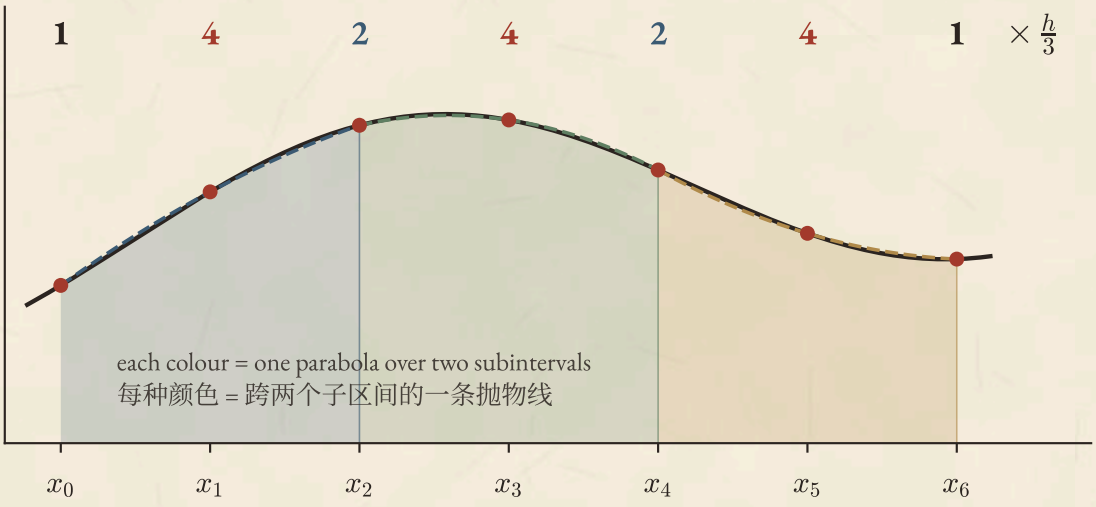


图 3 三条抛物线各管两个子区间；顶部数字是每个节点的权重（再乘 $h/3$）。共享的偶数节点被两条抛物线各用一次，所以权重 1+1=2。

Fig. 3 — One parabola per pair of subintervals; shared even nodes collect weight 1 + 1 = 2.

❖ 不用背拉格朗日：三行推出 $\frac{h}{3}(1,4,1)$ *A quick derivation*

推导 *Derivation*

平移使 $x_i = 0$，节点为 $-h, 0, h$。设抛物线 $P(x) = \alpha + \beta x + \gamma x^2$，则

$$\int_{-h}^{h} P\,dx = 2\alpha h + \frac{2\gamma h^3}{3} \quad (\beta \text{ 项是奇函数，积分为零}).$$

由插值条件：$\alpha = f_i$，且 $f_{i-1} + f_{i+1} = P(-h) + P(h) = 2\alpha + 2\gamma h^2$，即 $\gamma h^2 = \frac{1}{2}(f_{i-1} + f_{i+1}) - f_i$。代入：

$$\int_{-h}^{h} P\,dx = 2hf_i + \frac{2h}{3}\left[\tfrac{1}{2}(f_{i-1} + f_{i+1}) - f_i\right] = \frac{h}{3}(f_{i-1} + 4f_i + f_{i+1}). \blacksquare$$

❖ 精度：四阶，而且对三次多项式精确 *Fourth order — and exact for cubics*

泰勒分析给出：每对子区间局部误差 $-\frac{h^5}{90}f^{(4)}(\xi)$，全局误差 $-\frac{(b-a)h^4}{180}f^{(4)}(\xi)$，即**局部** $O(h^5)$、**全局** $O(h^4)$——讲义所说的 “fourth-order accuracy”。步长减半，误差除以 16。

彩蛋：抛物线本来只该对二次多项式精确，但三次项 x^3 在对称区间 $[-h, h]$ 上是奇函数、积分为零，于是辛普森对**不超过三次**的多项式都精确——又一次“对称性白送一阶”。

还有一个联系：$E_M \approx -\frac{1}{2}E_T$，所以加权平均 $\frac{2M+T}{3}$ 能把 h^2 主项消掉——这恰好就是辛普森公式（步长 $h/2$ 的版本）。用讲义数据验算：$\frac{2(2.0082484)+1.9835235}{3} = 2.0000068$。

> *Simpson = a cleverly weighted average of midpoint and trapezoid that cancels their h^2 errors: $S_{2n} = \frac{2M_n+T_n}{3}$.*

代码勘误 · 讲义第 9 页 *Bug in slide 9 code*

讲义的实现是 `2*sum(f[:n-2:2])`。当 `n = 11` 时，`f[:9:2]` 选中的下标是 **0, 2, 4, 6, 8**——把 `f[0]` 也算进去了！加上单独的 `f[0]`，端点实际权重变成 $1 + 2 = 3$。正确写法是 `f[2:n-2:2]`（下标 2, 4, 6, 8）。

w	**1**	4	2	4	2	4	2	4	2	4	**1**
	f[0]	f[1]	f[2]	f[3]	f[4]	f[5]	f[6]	f[7]	f[8]	f[9]	f[10]

```
f[1:n-1:2]  → odd 奇数下标 ×4
f[2:n-2:2]  → interior even 内部偶数下标 ×2   (NOT f[:n-2:2])
```

为什么讲义输出还是对的？因为 $f(x_0) = \sin 0 = 0$，多算的那份是零。换个函数就露馅：对 $\int_0^1 e^x\,dx$（11 点），讲义写法得 1.784949（误差 -6.7×10^{-2}，**比左黎曼和还差**），修正后得 1.7182828（误差 -9.5×10^{-7}）。

> *The slice `f[:n-2:2]` includes index 0, so f_0 gets weight 3 instead of 1. It goes unnoticed only because $\sin 0 = 0$. Use `f[2:n-2:2]`.*

批 f[0] 偷偷领了三份工资——只是它的工资恰好是零，所以财务没发现。

陆 实战：$\int_0^{\pi} \sin x\,dx = 2$

Worked example & convergence study

讲义的统一例题：11 个等距点（10 个子区间，$h = \pi/10$），与精确值 2 比较。下面是**整合后的修正版代码**（所有数字均已重新运行核对）：

```
import numpy as np

a, b, n = 0, np.pi, 11          # n = number of POINTS
h = (b - a) / (n - 1)           # 10 subintervals, h = pi/10
x = np.linspace(a, b, n)        # linspace INCLUDES both endpoints
f = np.sin(x)

I_L = h * np.sum(f[:-1])                        # left endpoints  x_0 .. x_9
I_R = h * np.sum(f[1:])                         # right endpoints x_1 .. x_10
I_M = h * np.sum(np.sin((x[:-1] + x[1:]) / 2))   # midpoints y_0 .. y_9
I_T = h/2 * (f[0] + 2*np.sum(f[1:-1]) + f[-1])
I_S = h/3 * (f[0] + 4*np.sum(f[1:-1:2]) + 2*np.sum(f[2:-1:2]) + f[-1])

for name, I in [("L", I_L), ("R", I_R), ("M", I_M), ("T", I_T), ("S", I_S)]:
    print(name, I, 2 - I)
```

方法 Method	近似值 Approx.	实际误差 $E = 2 - I$ actual	理论主项预测 predicted leading term	阶 order
左黎曼 Left Riemann	1.9835235375	+0.0164765	$\frac{h}{2}[f(b) - f(a)] = 0$ → 降级为二阶 drops to 2nd order	$O(h)^*$
右黎曼 Right Riemann	1.9835235375	+0.0164765	同上 same	$O(h)^*$
中点 Midpoint	2.0082484079	−0.0082484	$-\frac{h^2}{12} = -0.0082247$	$O(h^2)$
梯形 Trapezoid	1.9835235375	+0.0164765	$+\frac{h^2}{6} = +0.0164493$	$O(h^2)$
辛普森 Simpson	2.0001095173	−0.0001095	$-\frac{h^4}{90} = -0.0001082$	$O(h^4)$
`quad`	2.0	0.0	估计 est. 2.2×10^{-14}	adaptive

* 一般情形为一阶；本例因 $\sin 0 = \sin \pi$ 而“幸运地”表现为二阶。预测值由 $E_M \approx \frac{h^2}{24}[f'(b) - f'(a)]$、$E_T \approx -\frac{h^2}{12}[f'(b) - f'(a)]$、$E_S \approx -\frac{h^4}{180}[f'''(b) - f'''(a)]$ 计算，其中 $f'(\pi) - f'(0) = -2$，$f'''(\pi) - f'''(0) = 2$。理论与实际吻合到 1–2%。

考点 · 读表要点 *Reading the table*

- $\sin x$ 在 $[0,\pi]$ 上**凹**（$f''<0$）：梯形（弦在下方）**低估**，$E>0$；中点**高估**，$E<0$。
- 中点误差 $\approx$ 梯形误差的 $-\frac{1}{2}$：$-0.00825 \approx -\frac{1}{2}(0.01648)$。
- 同样 11 个点，辛普森比梯形准约 150 倍——成本几乎相同，只是换了权重。

> *Concave f ⇒ trapezoid under-estimates, midpoint over-estimates; Simson is ~150× more accurate for the same 11 samples.*

❖ 收敛性实验：用斜率读出阶数 *Reading the order off a log–log plot*

在双对数坐标中，$E \approx Ch^p$ 变成直线 $\log E = \log C + p\log h$，**斜率就是阶数** p。下图中 p 值由最后两点实测得到。

步长减半 halve h	黎曼 Riemann	中点 / 梯形 Mid / Trap	辛普森 Simpson
误差缩小倍数 error shrinks by	× 1/2	× 1/4	× 1/16
误差降到 1/100 需要点数约 points needed for 100× smaller error	× 100	× 10	× 3.2

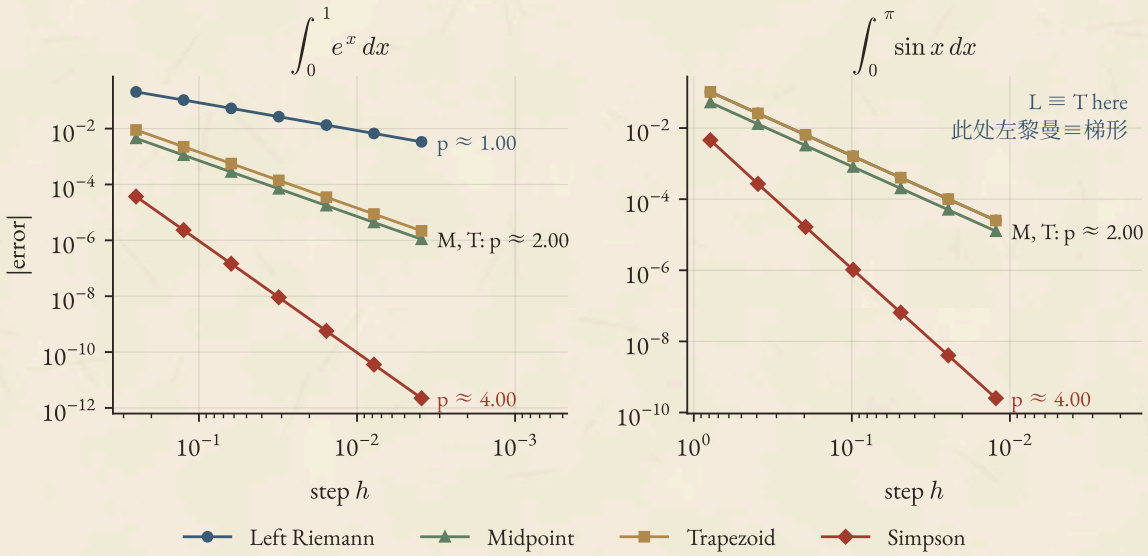


图 4 左：$\int_0^1 e^x\,dx$，各方法显出"本色"阶数 1、2、2、4。右：$\int_0^\pi \sin x\,dx$，左黎曼因端点值相等而与梯形完全重合、表现为二阶。

Fig. 4 — Measured slopes give the orders. On sin, *left Riemann coincides with the trapezoid rule and looks second-order — a lucky cancellation, not a property of the method.*

SciPy 工具箱

Numerical integration in Python

手写公式是为了理解；真正干活用 `scipy.integrate`。讲义介绍了三类工具：梯形积分、累积梯形积分、自适应求积。

版本提醒 *Version note — read before you run the slides*

讲义用的 `trapz`、`cumtrapz`（以及 `simps`）**已在 SciPy 1.14 中被移除**，新名字分别是 `trapezoid`、`cumulative_trapezoid`、`simpson`。我在 SciPy 1.18 / NumPy 2.5 上实测：旧名均不存在；NumPy 里的 `np.trapz` 也已让位于 `np.trapezoid`。另外 `plt.style.use('seaborn-poster')` 在新版 Matplotlib 中会报错，需改为 `'seaborn-v0_8-poster'`。

> *If you copy the slide code into a current environment it will fail on import. Old → new:* `trapz → trapezoid`, `cumtrapz → cumulative_trapezoid`, `simps → simpson`.

① `trapezoid`：数据的梯形积分 *Trapezoidal rule on samples*

```
from scipy.integrate import trapezoid, simpson

x = np.linspace(0, np.pi, 11)
y = np.sin(x)
print(trapezoid(y, x))      # NOTE the order: y first, then x
print(simpson(y, x=x))      # Simpson on the same samples
```

```
output ›  1.9835235375094544
2.0001095173150043
```

注意**参数顺序是 `(y, x)`**，写反不会报错但结果全错。与手写版本相比，末位数字的差别（…542 vs …546，量级 10^{-16}）来自浮点求和顺序不同的**舍入误差**，不是方法差异。

② `cumulative_trapezoid`：累积积分 *The running integral* $F(X) = \int_{x_0}^{X} f\,dx$

```
from scipy.integrate import cumulative_trapezoid

x = np.arange(0, np.pi, 0.01)                    # arange EXCLUDES the end: stops at 3.14
F  = cumulative_trapezoid(np.sin(x), x)          # length len(x) - 1  (no F(x_0))
F0 = cumulative_trapezoid(np.sin(x), x, initial=0) # length len(x), starts at 0
```

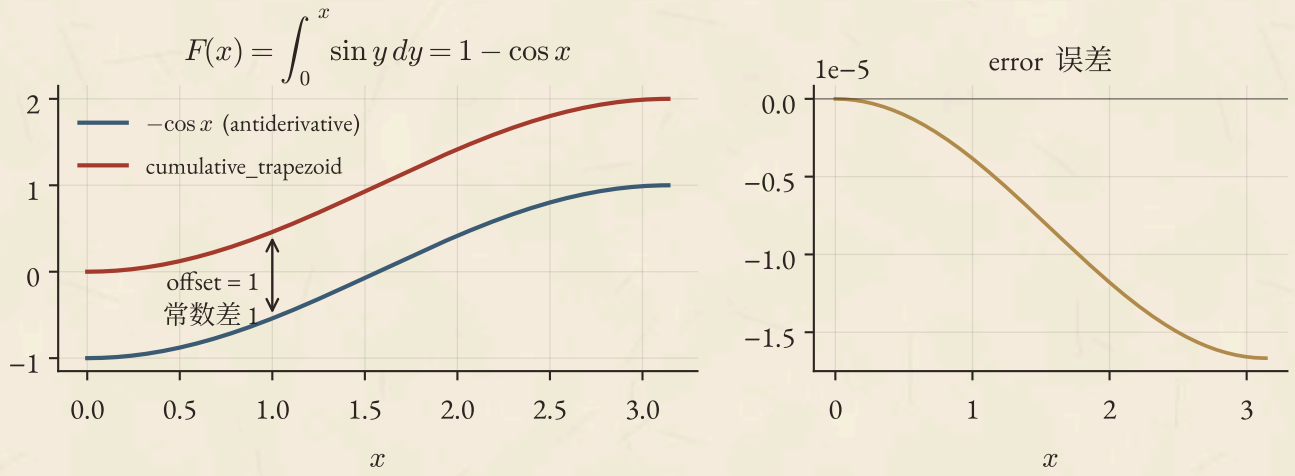


图 5 讲义图中的“Exact with Offset”是 $-\cos x$：它是一个原函数，但 $\int_0^x \sin y\,dy = 1 - \cos x$，两者相差常数 1。右：累积梯形积分的误差在 10^{-5} 量级。

Fig. 5 — $-\cos x$ is an antiderivative; the running integral from 0 is $1 - \cos x$, hence the constant offset of 1.

考点 · 三个细节 *Three details*

1. **长度少一：**默认输出比 `x` 短 1（没有 $F(x_0) = 0$ 这一项），所以讲义画图要写 `plt.plot(x[1::], F_approx)`。加 `initial=0` 则长度一致。
2. **原函数 vs 定积分：**“exact solution $F(x) = -\cos x$”只是差一个常数的原函数；从 0 积起的准确值是 $1 - \cos x$。
3. **`arange` 不含终点：** `np.arange(0, np.pi, 0.01)` 最后一个点是 3.14，不是 π，所以最终值约 1.99998 而非 2。要包含端点用 `linspace`。

❖③ `quad`：自适应求积 *Adaptive quadrature*

```python
from scipy.integrate import quad

I_quad, est_err = quad(np.sin, 0, np.pi)   # pass a FUNCTION, not an array
print(I_quad, est_err, 2 - I_quad)
```

```
output ›  2.0 2.220446049250313e-14 0.0
```

`quad` 接收的是**函数**（可调用对象），而不是数据数组。它在误差大的地方自动加密采样（底层为 QUADPACK 的高斯–克朗罗德规则），返回 **(积分值, 绝对误差估计)**。第二个数是**估计**，不是真实误差：这里估计 2.2×10^{-14}，实际为 0。

`quad` takes a **callable**, not samples. It refines adaptively where the integrand is difficult (QUADPACK, Gauss–Kronrod rules) and returns **(value, estimated absolute error)**. The second number is an *estimate* — here 2.2×10^{-14}, while the true error is 0.

批

quad 是“带保险的积分”：结果附赠一张误差估计单。至于保险公司赔不赔——通常赔，但对病态被积函数要看条款。

你手里有…… You have...	你想要…… You want...	用 Use
可调用的函数 a Python function	一个高精度数值 one accurate number	`quad(f, a, b)`
采样数据 sampled data (x, y)	一个数值 one number	`trapezoid(y, x)` / `simpson(y, x=x)`
采样数据 sampled data (x, y)	累积曲线 $F(x)$ running integral	`cumulative_trapezoid(y, x, initial=0)`
能在任意点求值 can evaluate anywhere	简单手写的二阶方法 a simple 2nd-order rule	中点法则 midpoint rule

捌 总表与易错点

Summary table & the examiner's favourite traps

方法 Method	每段图形 Local shape	权重 Weights	局部误差 Local error	全局 Global	精确至 Exact for deg ⩽	要求 / 备注 Requirement
左/右黎曼 Left / Right	矩形 rectangle	$h[1,\dots,1,0]$ $h[0,1,\dots,1]$	$\pm\frac{h^2}{2}f'$	$O(h)$	0	最简单 simplest
中点 Midpoint	矩形（中点高） rectangle	h at y_i	$+\frac{h^3}{24}f''$	$O(h^2)$	1	需中点值 needs $f(y_i)$
梯形 Trapezoid	直线 line	$\frac{h}{2}[1,2,\dots,2,1]$	$-\frac{h^3}{12}f''$	$O(h^2)$	1	适合数据 data-friendly
辛普森 Simpson	抛物线 parabola	$\frac{h}{3}[1,4,2,\dots,4,1]$	$-\frac{h^5}{90}f^{(4)}$ per pair	$O(h^4)$	3	奇数个点 odd # points
`quad`	自适应 adaptive	—	—	返回误差估计 returns estimate	—	需函数 needs callable

❖ 易错点清单 *Common pitfalls*

1. **点数 vs 区间数**。代码 `n = 11` 是点数 → 10 个区间，`h = (b-a)/(n-1)`。
   > *Points ≠ intervals.*
2. **局部 vs 全局阶**。全局阶 = 局部阶 − 1。题目问 "single interval" 就写局部阶。
   > *Global order is one less than local order.*
3. **梯形法则是二阶**。讲义第 7 页的 $O(h^2)/O(h)$ 有误，应为 $O(h^3)/O(h^2)$。
   > *Trapezoid: $O(h^3)$ local, $O(h^2)$ global.*
4. **辛普森偶数下标切片**。用 `f[2:n-2:2]`，不是 `f[:n-2:2]`；奇数下标 `f[1:n-1:2]`。
5. **辛普森需要奇数个点**。10 个点 = 9 个区间，无法两两配对；可以对前 9 个点用辛普森、最后一段另行处理（或用 3/8 法则、或用 SciPy 的 `simpson`，它对偶数个点有专门处理）。
6. **新版 API**。`trapz/cumtrapz/simps` 已移除；`trapezoid(y, x)` 参数顺序 y 在前。
7. `cumulative_trapezoid` **长度少 1**，除非 `initial=0`。
8. `arange` **不含终点，** `linspace` **含**。
9. `quad` **的第二个返回值是误差估计**，且 `quad` 需要函数而非数组。
10. $L = R = T$ **只是巧合**。源于 $f(a) = f(b)$，不能推广为"左右黎曼总是相等"。
11. **误差符号**。E = 精确 − 近似。$f'' < 0$（凹）时梯形 $E > 0$（低估），中点 $E < 0$（高估）；$f'' > 0$ 时相反。

玖 练习题

Practice problems — try before peeking

题型模仿考试：选择、手算、读代码、简答、推导。答案在下一节。*Exam-style mix: MCQ, hand computation, code reading, short answer, derivation.*

Q1 辛普森法则（步长 h）在整个区间上的误差阶是： 选择

The global error of composite Simpson's rule with step h is:

(A) $O(h)$ (B) $O(h^2)$ (C) $O(h^4)$ (D) $O(h^5)$

Q2 用梯形法则以步长 h 算得误差 3.2×10^{-3}。步长改为 $h/2$，误差约为： 选择

Trapezoid rule with step h has error 3.2×10^{-3}. With step $h/2$ the error is about:

(A) 1.6×10^{-3} (B) 8×10^{-4} (C) 4×10^{-4} (D) 2×10^{-4}

Q3 用节点 $x=0,1,2$（$h=1$）手算 $\int_0^2 x^2\,dx$ 的 L, R, M, T, S 及各自误差。精确值为 $8/3$。哪个是精确的？为什么？ 手算

By hand, with nodes $0,1,2$, compute L, R, M, T, S for $\int_0^2 x^2dx$ and their errors. Which one is exact, and why?

Q4 用 3 个点（$h=0.5$）计算 $\int_0^1 e^x\,dx$ 的梯形与辛普森近似（保留 6 位小数），并与精确值 $e-1\approx1.718282$ 比较。 手算

With 3 points ($h=0.5$) compute the trapezoid and Simpson approximations of $\int_0^1 e^x\,dx$.

Q5 设 `n = 11`。分别写出 `f[1:n-1:2]`、`f[2:n-2:2]`、`f[:n-2:2]` 选中的下标。讲义第 9 页的辛普森代码错在哪？为什么输出仍正确？ 读代码

List the indices selected by each slice. What is wrong with the slide's Simpson code, and why did it still print the right answer?

Q6 已有数组 `x`、`y`（`y = np.sin(x)`）。写出用 SciPy 计算 ① 定积分 和 ② 与 `x` 等长的累积积分数组 的两行代码。 写代码

Given arrays `x`, `y`, write one line for the definite integral and one for a running integral of the same length as `x`.

Q7 用英文简要解释：为什么中点法则比左黎曼和精确，尽管两者都只用矩形？ 简答

Explain briefly why the midpoint rule is more accurate than the left Riemann sum, even though both use rectangles.

Q8 证明梯形法则在单个子区间上的误差为 $-\frac{h^3}{12}f''(y_i)+O(h^5)$，并由此说明全局阶。 推导

Show that the single-interval trapezoid error is $-\frac{h^3}{12}f''(y_i)+O(h^5)$ and deduce the global order.

Q9 实验每 0.5 s 记录一次温度，共 20 个读数，需要积分。能否直接用中点法则？能否直接用复合辛普森法则？你会怎么做？ 应用

You have 20 temperature readings, one every 0.5 s. Can you use the midpoint rule? Composite Simpson? What would you do?

Q10 为什么讲义中 $\int_0^\pi \sin x\,dx$ 的左黎曼、右黎曼与梯形结果完全相同？对 $\int_0^1 e^x\,dx$ 是否仍然成立？ 概念

Why do the left, right and trapezoid results coincide for $\int_0^\pi \sin x\,dx$? Would they for $\int_0^1 e^x\,dx$?

Q11 `quad(np.sin, 0, np.pi)` 返回什么？第二个数是真实误差吗？能否把采样数组直接传给 `quad`？ 概念

What does `quad` return? Is the second value the true error? Can you pass it an array of samples?

❦ ❦ ❦

拾 参考答案 *Answers with reasoning*

A1 **(C)** $O(h^4)$。局部（每对子区间）$O(h^5)$，全局降一阶为 $O(h^4)$。(D) 是局部阶——典型干扰项。

A2 **(B)** 8×10^{-4}。梯形法则全局二阶，步长减半误差除以 $2^2=4$。若误信讲义的“$O(h)$”会错选 (A)。

A3 $f=x^2$：$f_0=0$，$f_1=1$，$f_2=4$；中点 $f(0.5)=0.25$，$f(1.5)=2.25$。

方法	计算	结果	误差 $\frac{8}{3}-I$
L	$1\cdot(0+1)$	1	$+5/3$
R	$1\cdot(1+4)$	5	$-7/3$
M	$1\cdot(0.25+2.25)$	2.5	$+1/6$
T	$\frac{1}{2}(0+2\cdot1+4)$	3	$-1/3$
S	$\frac{1}{3}(0+4\cdot1+4)$	$8/3$	0

辛普森精确：它对不超过三次的多项式都精确。另可验证 $E_M=-\frac{1}{2}E_T$ 恰好成立（因为 $f''\equiv2$ 为常数，误差主项就是全部误差），以及 $S=\frac{2M+T}{3}=\frac{5+3}{3}=\frac{8}{3}$。

A4 $f_0=1$，$f_1=e^{0.5}=1.648721$，$f_2=e=2.718282$。
$T=\frac{0.5}{2}(1+2\times1.648721+2.718282)=1.753931$，误差 -0.035649；
$S=\frac{0.5}{3}(1+4\times1.648721+2.718282)=1.718861$，误差 -0.000579。
同样三个点，辛普森误差约为梯形的 $1/60$。两者都高估（$E<0$），因为 e^x 是凸函数。

A5 `f[1:n-1:2]` → 1, 3, 5, 7, 9（奇数下标，权重 4）；`f[2:n-2:2]` → 2, 4, 6, 8（内部偶数下标，权重 2）；`f[:n-2:2]` → **0**, 2, 4, 6, 8。讲义用了最后一种，于是 f_0 的权重变成 $1+2=3$。输出仍正确是因为 $f_0=\sin0=0$；对 e^x 就会给出 1.784949 这样的错误结果。

A6

```
I = trapezoid(y, x)                          # or simpson(y, x=x)
F = cumulative_trapezoid(y, x, initial=0)  # same length as x
```

注意 `(y, x)` 的顺序，以及 `initial=0` 才能与 `x` 等长。

A7

> *"Expanding f in a Taylor series about the midpoint, the first-order term $f'(y_i)(x-y_i)$ is odd about y_i and integrates to zero over the symmetric subinterval. The leftover error is $\frac{h^3}{24}f''(y_i)$ per interval, so the midpoint rule is $O(h^2)$ globally, whereas the left Riemann sum keeps the $\frac{h^2}{2}f'(x_i)$ term and is only $O(h)$."*

中文要点：中点展开时一阶项是奇函数、积分为零，误差从 $O(h^2)$/段 提升到 $O(h^3)$/段；全局从一阶升为二阶。

A8 见第肆节推导：端点值在 y_i 处展开得 $\frac{f_i+f_{i+1}}{2} = f(y_i) + \frac{h^2}{8}f''(y_i) + O(h^4)$；精确积分为 $hf(y_i) + \frac{h^3}{24}f''(y_i) + O(h^5)$；相减得 $-\frac{h^3}{12}f''(y_i) + O(h^5)$。共 $(b-a)/h$ 段，全局 $O(h^2)$。

A9 **中点法则：不能直接用**——中点（如 0.25 s、0.75 s）没有测量值。**辛普森：不能直接覆盖全部**——20 个点是 19 个区间（奇数），无法两两配对。可行方案：① 直接用梯形法则（`trapezoid`）；② 对前 19 个点（18 个区间）用辛普森，最后一个区间单独用梯形；③ 用 SciPy 的 `simpson`，它对偶数个点会对最后一段做专门处理。

A10 因为 $R - L = h[f(b) - f(a)] = h(\sin\pi - \sin 0) = 0$，所以 $L = R$；又 $T = \frac{1}{2}(L+R)$，故三者相等。对 e^x：$f(1) - f(0) = e - 1 \neq 0$，三者互不相同。3 点时 $L = 1.324361$，$R = 2.183502$，$T = 1.753931$。

A11 返回元组 **(积分值, 绝对误差估计)** = (2.0, 2.22e-14)。第二个数是 QUADPACK 给出的**估计**，不是真实误差（真实误差这里是 0）。**不能**传数组：`quad` 需要能在任意点求值的函数；只有采样数据时用 `trapezoid` 或 `simpson`。

一页速记卡

One-page cheat sheet — 考前五分钟

四个公式 *Four rules*

$$L = h\sum_{i=0}^{n-1} f_i \qquad R = h\sum_{i=1}^{n} f_i$$

$$M = h\sum_{i=0}^{n-1} f(y_i),\ \ y_i = \tfrac{x_i + x_{i+1}}{2}$$

$$T = \tfrac{h}{2}\left[f_0 + 2\sum\nolimits_{\text{int}} f_i + f_n\right]$$

$$S = \tfrac{h}{3}\left[f_0 + 4\sum\nolimits_{\text{odd}} f_i + 2\sum\nolimits_{\text{even}} f_i + f_n\right]$$

误差阶 *Orders (local → global)*

Riemann	$\frac{h^2}{2} f'$	$O(h)$
Midpoint	$+\frac{h^3}{24} f''$	$O(h^2)$
Trapezoid	$-\frac{h^3}{12} f''$	$O(h^2)$
Simpson	$-\frac{h^5}{90} f^{(4)}$	$O(h^4)$

全局 = 局部 − 1。$h \to h/2$：误差 ÷ 2^p。

$E_M \approx -\frac{1}{2} E_T$，$S = \frac{2M+T}{3}$。

代码切片 *Slices (n = #points)*

- `h = (b-a)/(n-1)`
- 左 `f[:-1]`　右 `f[1:]`　内部 `f[1:-1]`
- 中点 `(x[:-1]+x[1:])/2`
- 奇 `f[1:n-1:2]`　偶 `f[2:n-2:2]`
- × `f[:n-2:2]` 含 `f[0]`

SciPy（新名） *Current names*

- `trapezoid(y, x)`　(旧 trapz)
- `simpson(y, x=x)`　(旧 simps)
- `cumulative_trapezoid(y, x, initial=0)`
- `val, est = quad(func, a, b)`
- `linspace` 含终点；`arange` 不含

七句话背完本章 *The chapter in seven lines*

① 切段 → 每段用简单图形 → 求和。
② 全局阶 = 局部阶 − 1。
③ 黎曼一阶；中点、梯形二阶；辛普森四阶。
④ 对称性消奇次项：中点升一阶，辛普森白送三次精确。
⑤ 中点需中点值；辛普森需奇数个点；quad 需函数。
⑥ 讲义勘误：梯形是 $O(h^3)/O(h^2)$；辛普森偶数切片从 2 开始。
⑦ $f(a) = f(b)$ 时 $L = R = T$——是巧合，不是定理。

1. Chop, approximate locally, sum.
2. Global order = local order − 1.
3. Riemann 1st; midpoint & trapezoid 2nd; Simpson 4th.
4. Symmetry kills odd terms — midpoint gains an order, Simpson is exact for cubics.
5. Midpoint needs midpoints; Simpson needs an odd number of points; `quad` needs a function.
6. Errata: trapezoid is $O(h^3)/O(h^2)$; Simpson's even slice starts at 2.
7. $f(a) = f(b) \Rightarrow L = R = T$ — coincidence, not a theorem.

❦ 据 PHYB57 第二十一章讲义整理；所有数值均以 NumPy 2.5 / SciPy 1.18 重新运行核对 ❦

Compiled from the C21 lecture slides; every number re-run and checked.